Higher Dynamic AES for Commercial Market*

Yi-Li Huang, Fang-Yie Leu†, Ruey-Kai Sheu, Shyh-Wei Chen, Kai-Yi Tan and Hua-Sheng Tseng
Department of Computer Science, Tunghai University, Taichung, 40764, Taiwan
{yifung, leufy, rickysheu, chensw, S0935070, S09350116} @thu.edu.tw
†Corresponding Author: yifung@thu.edu.tw; Tel.: +886-4-23590121

Abstract

Recently, Advanced Encryption Standard (AES) has been widely used in commercial markets. Auctions that compete expensive objects are typical examples. However, recent developments in cryptanalysis have revealed vulnerabilities, such as key-independent and static operations. In this study, AES AddRoundKey was identified as the only key-dependent operation, whereas the others were key independent operations. Hence, we proposed an enhanced version of AES called the Dynamic AES (DAES) to reinforce its commercial security. In addition, we designed an irreversible algorithm called Dynamic Shifting Mapping (DSM) to produce a dynamic box given a string. It has the key-dependent property of 64 variations to replace AES's ShiftRows. The results of our qualitative and efficiency analyses indicate that the DAES surpasses the AES in terms of security level with a slight trade-off in execution efficiency.

1. Introduction

Up to present, National Institute of Standards and Technology (NIST) has approved several cryptographic algorithms in which Advanced Encryption Standard (AES) (Dworkin, Turan, and Mouha, 2023) is crafted using Galois Field and composed of its Key Schedule and four round operations, including AddRoundKey, SubBytes, ShiftRows and MixColumns.

In fact, round operations of the AES can be classified as static which is independent of the Cipher Key (CK) or password (PW), meaning that its key-independent and dynamic properties are only determined by CK/PW. The AES using an identical substitution box (S-box) for all SubBytes in different rounds is key-independent. It takes the bias value as a constant to perform ShiftRows, utilizes an identical Maximum Distance Separable (MDS) matrix for all MixColumns operations in all rounds, and performs an XOR operation on the state and round key, called AddRoundKey, are key-dependent, implying that SubBytes, ShiftRows, and MixColumns are all static. Once the three operations are

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 30, July 8–10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

defined, they are used throughout the encryption process. On the other hand, AddRoundKey is dynamic since the 11 round keys are derived from CK/PW through the Key Schedule. Our conclusion is that AES is one with 1/4 dynamic.

Owing to the rapid development of software and hardware over the past two decades, Artificial Intelligence (AI) cracking by integrating a massive number of CPUs, GPUs, and NPUs (Yang, et al., 2023) or quantum computers (Yang, Zolanvari, Jain, 2023; DeCusatis, et al. 2024), will soon be possible.

Therefore, in this paper, a better AES, called Dynamic AES (DAES) is proposed. It dynamically creates key-dependent round operations to up-grade the AES's Shift-Row and replace S-Box with D-boxes while inheriting the advantages of AES.

The rest of this paper is organized as follows. Section 2 introduces work related to this study. Section 3 describes the architecture of the DAES. Qualitative analyses on the security and the performance of the DAES and AES are presented in Section 4. Section 5 concludes the study and discusses our future work.

2. Background and Related Works

In this session, some histories and security problems are described.

2.1 Threatens and Histories

AES is currently one of the most popular algorithms used for block-ciphering. The efficiency of the four abovementioned operations is high with complementary security functions. Therefore, this standard has been widely adopted to secure transmitted and stored data, particularly for wireless security (Nirbaya, et al. 2023), file encryption (Tong, et al., 2021), and processor security (Wang, et al. 2018). However, the AES has been partially cracked. Some of these issues are discussed below:

Wang et al. (Wang, et al. 2016) combined Fault Sensitivity Analysis (FSA) and a zero-value sensitivity model to break the masked AES S-box. The authors demonstrated that traversing the zero-valued input of the S-box can ultimately reveal the last-round key of AES. In (Jithendra and Shahana, 2023), Jithendra and Shahana conducted a biclique at-tack on a full-round PRESENT-80 block cipher using independent related key differential cryptanalysis. The authors claimed that the computational complexity of such an attack was significantly reduced, thus endangering AES.

2.2 Enhancing S-box Security

To enhance the security of AES, many researchers have proposed new features such as randomness, dynamicity, and key dependency. Assafli and Hashim (Assafli, and Hashim, 2020) introduced a dynamic box that constantly changes the independence of the encryption key. A new S-box is generated depending on the epoch timestamp to produce an identical nonlinear S-box inside the transmitter and receiver, thus enhancing the security level of the cryptography system. The authors claimed that data complexity and time complexity were 223 and 279.63, respectively.

Abd-El-Atty (Abd-El-Atty, 2013) proposed a new S-box mechanism based on quantum-inspired quantum walks, the Hénon map, and a customized particle swarm optimization algorithm. The performance analysis shows that the effectiveness and reliability of designing various cryptosystems are excellent. With S-box, a new image cryptosystem was developed to increase its efficacy and security against various attacks.

Saha et al. (Saha, et al., 2018) proposed a novel approach for generating key-dependent S-boxes using a pseudo-random methodology to enhance the secrecy of the S-boxes.

2.3 Other Recent Enhancements

Building upon these studies, Luong and Linh (Luong and Linh, 2014) proposed new key-dependent XOR tables to strengthen AES security and evaluated the randomness of the block cipher outputs. Kazlauskas (Kazlauskas, 2015) enhanced the confusion property and avalanche effect of AES by invoking a Symmetric Random Function Generator.

Su et al. (Su, et al., 2019) researched data encryption standards based on the AES algorithm in an IoT environment, showing applicability of AES in emerging technological fields.

In summary, improving the AES Key Schedule or static operations is crucial for enhancing security. These findings indicate that AES can significantly improve its resistance to differential and linear cryptanalysis without substantially affecting its encryption/decryption performance by introducing features, such as randomness, dynamicity, and key dependency. However, the trade-off between the execution performance and security level remains a critical re-search topic. In other words, AES requires continuous improvement and enhancement to meet the growing security demands.

3. The Architecture of the DAES

The DAES was developed following the AES framework with its Key Schedule and transformation rounds being expanded, including the generation of 11 RoundKeys, 11 D-boxes, and 20 round parameters, to strengthen the security level of each transformation round, attempting to improve its encryption/decryption throughputs for commercial market. The transformation rounds is modified as follows. (1) generating a dynamic box (D-box) based on user CK/PW to substitute for the S-box which is a static box, thus enhancing the security level of the SubBytes operation; (2) proposing a DBR operation which provides 64 variations for data encryption/decryption. In fact, in the AES the ShiftRows operation is one of these variations. The original AES ShiftRows operation is just one of the DBRs. Originally, the AES had approximately 1/4 of dynamic transformation rounds. After our modifications, the DAES is a 3/4 dynamic cryptographic system.

1.1 Parameters

The parameters used in this study are defined below:

1. D-box: A 16 × 16 (D-box) is derived from user input strings. The 256 non-duplicated elements in the S-box, ranging from 00 to FF, were randomly rearranged in the D-box.
2. ISKs: Initial Serial Keys (ISKs), including ISK_1 and ISK_2, are produced using a specific algorithm, given an input string. The length of the ISK is 1024 bits.
3. DKs: Dynamic Keys, including DK1, DK2 and DK3, are derived from ISK1 and ISK2 by employing 3D operations (Huang, et al., 2017] and the DSM algorithm. The length of the DK is 1024 bits.
4. FA: Flag Array that identifies the elements of the D-box is used to guarantee that each element of the D-box is unique without colliding with others. The indexed content of FA is "T" or "F," which indicates that the index has been selected or not, respectively.
5. $SIA1$: The 1st Insert Array is derived from DK1 by removing duplicated elements in DK1 using FA.
6. $SIA2$: The 2nd Insert Array is derived from DK2 and DK3 by removing the duplicated elements in $SIA1$, DK2 and DK3 using FA.
7. RIA: Residual Insert Array consists of those unselected indexes of FA.
8. $SIA3$: The 3rd Insert Array, which is obtained by rearranging the Residual Insert Array (RIA).
9. $SIA1$, $SIA2$, and $SIA3$ are used to establish the D-box.

10. RLs: Real lengths of the arrays, including S$IA1$, $SIA2$, $SIA3$, and RIA.
11. GBT1: "Generating a Box of Type 1," which is generated to produce a D-box by invoking a unique string.
12. GBT2: "Generating a Box of Type 2," which is produced to yield a D-box by invoking a key and initial box.

3.2 Operations

Let p_i (c_i) be the plaintext (ciphertext) block and let k be the encryption key. The encryption and decryption processes of the three fundamental operators are defined as follows.

(1) Exclusive-OR operator ($\oplus$):
Encryption: $c_i = p_i \oplus k$;
Decryption: $p_i = c_i \oplus k$;

(2) Binary addition operator ($+_2$):

Encryption: $c_i = p_i +_2 k$, where pi and k undergo binary addition, and the carry generated by the addition of the most significant bit is ignored.

Decryption: $p_i = c_i -_2 k = \begin{cases} c_i - k, & if\ c_i \geq k \\ c_i + \bar{k} + 1, & if\ c_i < k \end{cases}$ where $-_2$ is the inverse operation of $+_2$.

(3) Rotate-Equivalence operator ($\odot_R$):

Encryption: $c_i = p_i \odot_R k = p_{iR} \odot k$, where p_{iR} is the key obtained by rotating plaintext p_i clockwise $|k|/4$ bits. For example, if $|k| = 128$, p_i is rotated by 32 bits.

Decryption: $p_i = c_i \odot_{IR} k$ = counter_clockwise rotating ($c_i \odot k$), a total of $|k|/4$ bits.

3.3 Proposed Algorithms

In the following, we described the algorithms created and used in the DAES.

3.3.1 DSM Algorithm

DSM Algorithm with (P, D-box) as its arguments is shown below.

Algorithm 1: The DSM Algorithm
Input: A plaintext $P = p_1 p_2 ... p_k$ of n bits, where n is a multiple of 8, $k = n/8$, p_i is 8 bits long, $1 \leq i \leq k$, and a 16 × 16 D-box.
Output: Ciphertext $C = c_1 c_2 ... c_k$ where c_i is 8 bits long and $1 \leq i \leq k$.
1.Let dsn = 256; /* setting initial value of dsn, the dynamic shift number */
2. **for** i = 1 to k
    $vp[i]$ = Int(p_i); /* Int(p_i) retrieves ASCII code of p_i */
    $dsn = dsn + vp[i] * i$;
3. **next** i;
4. $vp[0] = vp[1] + vp[k] + dsn$;
5. $vp[k+1] = vp[k-1] + vp[k]$;
6. **for** i = 1 to k
7. $dsn = (dsn + vp[i-1] * vp[i] + vp[i+1]) \bmod 2^{16}$;
    $vch = (vp[i] + dsn) \bmod 256$;
    c_i = the ith ciphertext that is the content in the D-box at position vch;
8. **next** i;
9. **return** C;

In this algorithm, a character in the plaintext string is encrypted sequentially and individually. When encrypting a character, a new dsn is generated for encrypting the next character. In each encryption, the dsn is increased by multiplying and summing the ASCII codes of the previous, current, and next plaintext characters. This increment is accumulated in the dsn. Therefore, a change to any character in the plaintext will conduct subsequent changes to the ciphertext. The dsn value that encrypts the first character is the sum of the ASCII codes of all plaintext characters plus 256. Therefore, it has an excellent avalanche effect.

3.3.2 The DSM Expansion Algorithm

In this study, the length of the user input *CK/PW* is 128–1024 bits. The length of our encryption/decryption key was 1024 bits. When the length of the inputting key *CK/PW* is shorter than 1024 bits, a key expansion method called the DSM Expansion (DSM-E) algorithm, that is, Algorithm 2 as shown below, is invoked to produce a new *CK/PW* that is 1024 bits long. Of course, as shown in step 5 of this algorithm, when $|PW|$= 1024, PW is encrypted to securely protect this *PW*.

Algorithm 2: DSM-E Algorithm
Input: *PW*, where $|PW| \leq 1024$; A 16x16 box;
Output: An expanded *PW*, where $|PW| = 1024$;

1. **While** $|PW| < 1024$ do
   P' = DSM (*PW*, D-box);
   $PW = P' || PW || P'$;
2. **end while**
3. *PW* = DSM (*PW*, D-box); /* when $|PW| = 1024$, *PW* is encrypted */
4. **return** Right (*PW*, 128); /* return PW of 1024 bits, i.e., 128 bytes */

3.3.3 Producing a Dynamic Box of 16 x 16

Dynamic Boxes are adopted to replace S-Box. The process of generating a 16 × 16 Dynamic Box (16DB) is composed of two processes: the Generation of Dynamic Keys using *PW* (GDK, i.e., Algorithm 3) and Dynamic Box Generator (DBG, i.e., Algorithm 4).

Between step 1 and step 11 in the GDK Algorithm (Algorithm 3), the input *PW* is reproduced as a new 1024-bit scrambled string ISK_1 by using DSM-E Algorithm (Algorithm 2). Between step 12 and step 15 of Algorithm 3, ISK_1 is further scrambled and diffused by repeatedly using the DSM Algorithm (Algorithm 1), generating ISK_2, DK_1, DK_2 and DK_3, in which the latter three are utilized to produce a dynamic box, i.e., D-box, by invoking Algorithm 4.

Algorithm 3: GDK Algorithm
Input: PW; A 16x16 I-box;
Output: DK_1, DK_2 and DK_3;
1.If $|PW| < 1024$, then
2. ISK_1 = DSM-E (PW, I-box); /*$|ISK_1|$=1024 bits*/
3. else if $|PW| = 1024$, then
4. ISK_1 = DSM (PW, I-box);
5. else /* $|PW| > 1024$ */
6. $PW = p_1p_2p_3 \ldots p_{m-1}p_m$, where $|p_i| = 1024$, $1 \leqq i < m$, and $m \geqq 2$; /* partition PW into blocks, each with 1024 bits long */
7. If $|p_m| < 1024$, then /* the last block p_m may be shorter than 1024 bits*/
8. p_m = DSM-E (p_m, I-box);
9. end if
10. $ISK_1 = p_1 \oplus p_2 \oplus \ldots \oplus p_{m-1} \oplus p_m$;
11. end if
12. ISK_2 = DSM (ISK_1, I-box);
13. DK_1 = DSM($ISK_1 \oplus ISK_2$, I-box);
14. DK_2 = DSM($ISK_1 +_2 ISK_2$, I-box);
15. DK_3 = DSM($ISK_1 \odot_R ISK_2$, I-box);
16. **return** DK_1, DK_2 and DK_3;

Algorithm 4: The DBG Algorithm
Input: IK_1, IK_2 and IK_3; /* IK_1, IK_2 and IK_3 are DK_1, DK_2, DK_3*/
Output: D-box;
FA[256], D-box[256], /* Flag Array, and D-box array */
$SIA1$[128], $SIA2$[256], $SIA3$[256], RIA[256];
/* Insert Arrays and the residual Insert Array */
Let $AL1$=0; $AL2$=0; ALR=0; $AL3$=0; /* ALj denotes the actual length of $SIAj$, $1 \leq j \leq 3$ */
/* Set up initial arrays, variables, and parameters */
Let $IK_1 = KA_0||KA_1||\ldots||KA_{127}$, where $|KA_j| = 8$ (bit) for all j, $0 \leqq j \leqq 127$;
Let $IK_2 = KB_0||KB_1||\ldots||KB_{127}$, where $|KB_j| = 8$ (bit) for all j, $0 \leqq j \leqq 127$;
Let $IK_3 = KC_0||KC_1||\ldots||KC_{127}$, where $|KC_j| = 8$ (bit) for all j, $0 \leqq j \leqq 127$;
/* individually partition IK_1, IK_2 and IK_3 into 128 eight bits string*/
for j = 0 to 255
$FA[j]$ = "F";
next j;
for j = 0 to 127
$KA[j]$ = Int (KA_j); /* Int (KA_j) produces ASCII code of character KA_j */
$KB[j]$ = Int (KB_j);
$KC[j]$ = Int (KC_j);
next j;
/* Generate arrays $SIA1$, $SIA2$, and RIA */
for i = 0 to 127 /* filling in $SIA1$ according to the data in IK_1 */
if ($FA[KA[i]]$ = "F") then
$SIA1[AL1]$ = $KA[i]$;
$FA[KA[i]]$ = "T";
$AL1$ = $AL1$ + 1;
end if
next i;
for i = 0 to 127 / * filling in $SIA2$ based on data from IK_2 */
if ($FA[KB[i]]$ = "F") then
$SIA2[AL2]$ = $KB[i]$;

```
            FA[KB[i]] = "T";
            AL2 = AL2 + 1;
        end if
    next i;
    for i = 0 to 127 /* filling in SIA3 based on data from IK3 */
        if (FA[KC[i]] = "F") then
            SIA3[AL3] = KC[i];
            FA[KC[i]] = "T";
            AL3 = AL3 + 1;
        end if
    next i;
    for i = 0–255 /* considering the unused indices of FA as elements of RIA*/
        if (FA[i] = "F") then
            RIA[ALR] = i;
            ALR = ALR+1;
        end if
    next i;
        /* expanding SIA2 by concatenating RIA and SIA1*/
    for i = 0 to ALR–1 /* expanding SIA2 by concatenating RIA */
        SIA2[AL2+i] = RIA [i] ;
    next i;
    for i = 0 to AL1–1 /* expanding SIA2 by concatenating SIA1 */
        SIA2[AL2+ ALR +i] = SIA1 [i] ;
    next i;
        /* deriving parameters h1, h2, h3, jup1, jup2, and jup3 from arrays KB and KC */.
    h1=0; h2=0; h3=0; jup1=0; jup2=0; jup3=0;
    for j = 0 to 63
        h1= h1 + KB[j];
        jup1= jup1 + KB[64+j];
    next j
    for j = 0 to 31
        h2= h2 + KC[j];
        h3= h3 + KC[32+j];
        jup2 = jup2 + KC[64+j];
        jup3 =jup3 + KC[96+j];
    next j
    h1 = h1 mod 256;
    h2 = h2 mod 256;
    h3 = h3 mod 256;
    jup1= jup1 mod 128;
    if (jup1 is even) then
        jup1= jup1 +129;
    end if
    jup2= jup2 mod 128;
    if (jup2 is even) then
        jup2= jup2 +129;
    end if
    jup3= jup3 mod 128;
    if (jup3 is even) then
        jup3= jup3 +129;
    end if
        /* generate D-box using SIA2 and parameters h1, h2, h3, jup1, jup2, and jup3 */
```

for i = 0 to 255 /* rearranging $SIA2$ to obtain $SIA3$ */
$SIA3[i] = SIA2[(h_1 + i * jup1) \ mod \ 256]$;
next i;
for i = 0 to 255 /* generating D-box */
D-box[$(h_3 + i * jup3) \ mod \ 256$] = $SIA3[(h_2 + i * jup2) \ mod \ 256]$;
next i;
return D-box;

The DBG algorithm adopts the ASCII of CK/PW as its input key and involves DK_1, DK_2 and DK_3 to generate the elements of the D-box. The Flag Array FA is to make sure that a D-Box has no duplicate elements. At last, $SIA3$ is rearranged to break the order of the elements in the Residual Insert Array (RIA) to improve the randomness of $SIA3$.

The process of the generation of the D-box is summarized below: (1) Algorithm 3: The GDK Algorithm derives DK_1, DK_2 and DK_3 from the CK/PW. (2) Algorithm 4: The DBG Algorithm generates the elements of the D-box following the content order of these three dynamic keys. Since DK_1, DK_2 and DK_3 are generated by using uses DSM and DSM-E several times, it inherits the garbled text diffusion and input sensitivity. Thus, the elements of the D-box are completely determined by the CK/PW.

Algorithm 5: The GBT1 Algorithm
Input: PW; A 16 x 16 I-box;
Output: D-box;
1. GDK (PW, I-box); /* producing DK_1, DK_2, and DK_3 */
2. Call DBG (DK_1, DK_2, DK_3); /* generating D-box */
3. **return** D-box;

Algorithm 6: The GBT2 Algorithm
Input: PW; A 16x16 I-box;
Output: D-box;
1. $PW_1 = PW_0 || PW_1 || \ldots || PW_{15}$, where $|PW_j| = 8$ (bits) for all j, $0 \leqq j \leqq 15$;
/* logically partition PW_1 into 16 eight bits string*/
2. for i = 0 to 15
3. $PW[i]$ = int(PW_i);
4. next i;
5. h_1=0; h_2=0; $jup1$=0; $jup2$=0;
6. for i = 0 to 3
7. $h_1 = h_1 + PW[4i]$;
8. $h_2 = h_2 + PW[4i+1]$;
9. $jup1 = jup1 + PW[4i+2]$;
10. $jup2 = jup2 + PW;4i+3]$;
11. next i
12. $h_1 = h_1 \ mod \ 256$;
13. $h_2 = h_2 \ mod \ 256$;
14. $jup1 = jup1 \ mod \ 128$;
15. if ($jup1$ is even) then $jup1 = jup1 + 129$;
16. $jup2 = jup2 \ mod \ 128$;
17. if ($jup2$ is even) then $jup2 = jup2 + 129$;
18. for i = 0 to 255 /* filling in D-box according to the data in I-box */
19. D-box[$(h_2 + i * jup2) \ mod \ 256$] = I-box [$(h_1 + i * jup1) \ mod \ 256$];
20. next i;
21. **return** D-box;

Next, the GBT1 Algorithm (Algorithm 5) derives D-box from CK/PW. To accelerate the generation of the D-box, we first transform a default initial D-box, denoted by I-Box, into another D-box based on the PW. The GBT2 algorithm (Algorithm 6) uses PW to produce 4 parameters, h_1, h_2, $jup1$, and $jup2$, in which h_1 is the position of the first element removed from the I-box, h_2 is the position of the first element inserted into the produced D-box, $jup1$ is the offset for each movement in the I-box, and $jup2$ is the offset for each insertion in the produced D-box. $jup1$ and $jup2$ are odd integers. The GBT2 algorithm is shown below.

3.3.4 Dynamic Byte Rotation Operation

ShiftRows operation is a method that shifts its position of the same distance within the plaintext state block. In AES, without ShiftRows, it would be our sets of 32-bit encryption system. In fact, ShiftRows makes AES a 128-bit system, thus, enhancing the security level of ShiftRows. Thus, in this study, we develop a DBR algorithm to substitute for ShiftRows. In the DBR, 64 forms of shifting character position within the plaintext state block obey the above shifting rule. The original AES ShiftRows operation is one of the DBRs with index 16. The DBR rotates plaintext P with rotation index n derived from the Round Key using the following method, $0 \le n \le 63$. Its rotation has three types:

Let plaintext $P = p_0p_1p_2p_3\ldots p_{14}p_{15} = w_0w_1w_2w_3$, where $|p_j| = 1$ (byte), $0 \le j \le 15$, $|w_j| = 1$ (word), $0 \le j \le 3$, and the corresponding state be:

$$st = \begin{bmatrix} p_0 & p_4 & p_8 & p_C \\ p_1 & p_5 & p_9 & p_D \\ p_2 & p_6 & p_A & p_E \\ p_3 & p_7 & p_B & p_F \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ r_2 \\ r_3 \end{bmatrix}.$$

Type 1. $0 \le n \le 15$: Clockwise rotation of P for n bytes. /* The first 16 bytes */

Type 2. $16 \le n \le 39$: There are 0, 1, 2, and 3 rotating counts on each row in st, generating 4! permutations of 0, 1, 2 and 3, i.e., 0123, 0132, ..., 3201, 3210.

Type 3. $40 \le n \le 63$: Repeat Type 2. The state matrix st is substituted by the transpose matrix st^t.

For example, when n = 16, i.e., Type 2, the corresponding permutation is 0123, and the corresponding rotation on st is that r0 has no rotation, ri rotates i byte(s), $1 \le i \le 3$. This was the ShiftRows operation of the AES. The second example is that if n = 38 (also type 2), the corresponding permutation is 3201. The corresponding rotation is that r0 rotates three bytes, r_1 rotates two bytes, r_2 has no rotation, and r_3 rotates one byte.

Following the three types of P, we establish a DBR lookup table called the DBRTable, i.e., Table I, which keeps the possible DBR operation (see Algorithm 7) and can effectively enhance the efficiency of the DBR operation. The element in the DBRTable is the index sequence of ciphertext C corresponding to plaintext P after the DBR operation with rotation index n. For example, if $P = p_0p_1p_2p_3\ldots p_Ep_F$, then

DBR(P, 9) = $p_9p_Ap_Bp_Cp_Dp_Ep_Fp_0p_1p_2p_3p_4p_5p_6p_7p_8$, with index sequences of
9,*A*,*B*,*C*,*D*,*E*,*F*,0,1,2,3,4,5,6,7,8, i.e. subscripts, respectively.

DBR(P,25)=$p_4p_9p_Ep_3p_8p_Dp_2p_7p_Cp_1p_6p_Bp_0p_5p_Ap_F$, with index sequence of
4,9,*E*,3,8,*D*,2,7,*C*,1,6,*B*,0,5,*A*,*F*.

DBR(P,49)=$p_1p_6p_Bp_Cp_2p_7p_8p_Dp_3p_4p_9p_Ep_0p_5p_Ap_F$, with index sequence of
1,6,*B*,*C*,2,7,8,*D*,3,4,9,E,0,5,*A*,*F*.

n	Type 1	*n*	Type 2	*n*	Type 3
0	0,1,2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*,*F*	16	0,5,*A*,*F*,4,9,*E*,3,8,*D*,2,7,*C*,1,6,*B*	40	0,5,*A*,*F*,1,6,*B*,*C*,2,7,8,*D*,3,4,9,*E*
1	1,2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*,*F*,0	17	0,5,*E*,*B*,4,9,2,*F*,8,*D*,6,3,*C*,1,*A*,7	41	0,5,*B*,*E*,1,6,8,*F*,2,7,9,*C*,3,4,*A*,*D*
2	2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*,*F*,0,1	18	0,9,6,*F*,4,*D*,*A*,3,8,1,*E*,7,*C*,5,2,*B*	42	0,6,9,*F*,1,7,*A*,*C*,2,4,*B*,*D*,3,5,8,*E*
	…		…		…
3	*D*,*E*,*F*,0,1,2,3,4,5,6,7,8,9,*A*,*B*,*C*	37	*C*,5,*A*,3,0,9,*E*,7,4,*D*,2,*B*,8,1,6,*F*	61	3,5,*A*,*C*,0,6,*B*,*D*,1,7,8,*E*,2,4,9,*F*
4	*E*,*F*,0,1,2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*	38	*C*,9,2,7,0,*D*,6,*B*,4,1,*A*,*F*,8,5,*E*,3	62	3,6,8,*D*,0,7,9,*E*,1,4,*A*,*F*,2,5,*B*,*C*
5	*F*,0,1,2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*	39	*C*,9,6,3,0,*D*,*A*,7,4,1,*E*,*B*,8,5,2,*F*	63	3,6,9,*C*,0,7,*A*,*D*,1,4,*B*,*E*,2,5,8,*F*

Table I: The DBRTable storing rotation index n and its sequence for plaintext $P = p_0p_1p_2p_3\ldots p_Ep_F$

Algorithm 7: DBR Operation for Encryption
Input: Plaintext P, where $|P| = 16$ (bytes); rotation index n, $0 \le n \le 63$;
Output: Ciphertext C, where $|C| = 16$(byte)
1. Let $P = p_0 \| p_1 \| p_2 \| p_3 \| p_4 \| \ldots\ldots \| p_{12} \| p_{13} \| p_{14} \| p_{15}$;
   /* logically partition PW_1 into 16 eight bits string*/
2. For $i = 0$ to 15
3. $P[i]$ = int(p_i); /* extracting ASCII value of p_i */
4. $C[i] = P$[DBRTable(n, i)]; /* extracting element i of DBR using DBRTable(n, i) */
5. c_i = Char($C[i]$);
6. next i;
7. $C = c_0 \| c_1 \| c_2 \| c_3 \| c_4 \| \ldots\ldots \| c_{12} \| c_{13} \| c_{14} \| c_{15}$;
8. **return** C;

n	Type 1	*n*	Type 2	*n*	Type 3
0	0,1,2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*,*F*	16	0,*D*,*A*,7,4,1,*E*,*B*,8,5,2,*F*,*C*,9,6,3	40	0,4,8,*C*,*D*,1,5,9,*A*,*E*,2,6,7,*B*,*F*,3
1	*F*,0,1,2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*,	17	0,*D*,6,*B*,4,1,*A*,*F*,8,5,*E*,3,*C*,9,2,7	41	0,4,8,*C*,*D*,1,5,9,6,*A*,E,2,*B*,*F*,3,7
2	*E*,*F*,0,1,2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*	18	0,9,*E*,7,4,*D*,2,*B*,8,1,6,*F*,*C*,5,*A*,3	42	0,4,8,*C*,9,*D*,1,5,*E*,2,6,*A*,7,*B*,*F*,3
	…		…		…
13	3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*,*F*,0,1,2	37	4,*D*,*A*,3,8,1,*E*,7,*C*,5,2, *B*,0,9,6,*F*	61	4,8,*C*,0,*D*,1,5,9,*A*,*E*,2,6,3,7,*B*,*F*
14	2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*,*F*,0,1	38	4,9,2,*E*,8,*D*,6,3,*C*,1,*A*,7,0,5,*E*,*B*	62	4,8,*C*,0,9,*D*,1,5,2,6,*A*,*E*,*F*,3,7,*B*
15	1,2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*,*F*,0	39	4,9,*E*,3,8,*D*,2,7,*C*,1,6, *B*,0,5,*A*,*F*	63	4,8,*C*,0,9,*D*,1,5,*E*,2,6,*A*,3,7,*B*,*F*

Table II: The InvDBRTable storing rotation index n and its sequence for ciphertext $C = c_0c_1c_2c_3\ldots c_Ec_F$

Table II lists the results of the inverse DBR operation (see Algorithm 8).

Algorithm 8: DBR Operation for Decryption
Input: Ciphertext C, where $|C| = 16$ (byte),
rotation index n,where $0 \le n \le 63$;
Output: Plaintext P, where $|P| = 16$ (byte); /* divide C from 1024 bits into 128 bytes */
1. Let $C = c_0 \| c_1 \| c_2 \| c_3 \| c_4 \| \ldots\ldots \| c_{12} \| c_{13} \| c_{14} \| c_{15}$; /* logically partition*/
2. for $i = 0$ to 15
3. $C[i]$ = int(c_i); /* extracting ASCII value of c_i */
4. $P[i] = C$[InvDBRTable(n, i)]; /* extraction of element i using InvDBRTable(n, i) */
5. p_i = Char($P[i]$);
6. next i;
7. $P = p_0p_1p_2\ldots\ldots p_{14}p_{15}$;
8. **return** P;

4. Security and Qualitive Analyses

AES utilizes a reversible Key Schedule, making it vulnerable. But the DAES utilizes an irreversible Key and Parameter Schedules, which provides additional security features compared to AES. However, the four different operations in the AES round are interconnected and affect the overall security; a change in one will result in different ciphertexts. Therefore, quantitative analysis is not feasible in this paper. Next, we will use a qualitative analysis approach for discussion.

4.1 The Security on Operations of Transformation Rounds

The DAES framework follows that of the AES. Hence, the security analyses of the three dynamic operations (SubBytes, ShiftRows, and AddRounkey) follow the following rules.

(1) **SubBytes:** (A) For the same look-up operation, both the AES and DAES have the same operational security levels; (B) The S-box of AES is fixed and well known. Hence, the effective security length of the variable (ESLV for short) of the S-box is zero where the ESLV is defined as the security level of a variable. However, with the DAES, the 10 D-boxes used in each round (D_1-box–D_{10}-box) are derived from the D_0-box and RK_0. Both are all unknown to users, meaning that the ESLV of these 10 D-boxes is 1953 (= 1683 × 1 + 30 × 9) bits, where 1683 is the ESLV of the D_1-box and 30 is the ESLV of each Di-box, $2 \le i \le 10$.

In the DAES, the D-box is produced following the CK/PW. Without knowing the CK/PW, it is impossible for an attacker to acquire the D-box. Instead, guessing the D-box's contents is the probable method.

Claimed 1: If the original D-box is known and the RK is 128 bits long, then the ESLV of the D-box. derived from GBT2 is 30 bits.

Proof:

With GBT2, the D-box derived from the original D-box is determined by h_1, h_2, jup_1 and jup_2, $0 \le h_1, h_2 \le 255$ and $0 \le jup_1, jup_2 \le 127$. The possible selection of them is 2^{30} (= 256 × 256 × 128 × 128); Thus, the ESLV of the D-box derived by GBT2 is 30 bits. (Q.E.D.)

(2) **ShiftRows:** The ShiftRows operation in AES in each transformation round uses the same version, whereas the number of choices in the DBR is 64 in each transformation round and 64^r for each encryption, where r is the number of transformation rounds. In the DAES, r=10.

(3) **AddRoundKey**: AddRoundKey operations in the AES and DAES are identical. The ESLV is 128 bits. In the AES, once RK_0 is produced, the following RKs is then determined, i.e., having no contribution to ESLV, except RK_0. In the DAES, RK_i is derived from RK_{i-1} and D_i-box. If RK_0 is known, the following RKs are still unknown because it lacks D_i-box, $1 \le i \le 10$, i.e., the ESLV of AddRoundKey in the DAES is 128 × 11 = 1408 bits.

4.2 OSS and ESLV of the AES and DAES

Generally, the AES and DAES frameworks are similar. The operational security strength (OSS) and ESLV are two characteristic indicators and the SubBytes, ShiftRows, and AddRoundKey, in each transformation round of the AES are defined as operational security strengths of 1. Table III shows the results of the qualitative analysis.

Indicator / Operation	AES		DAES	
	OSS	ESLV (bits)	OSS	ESLV (bits)
SubBytes	1	0	1	1953
ShiftRows/DBR	1	0	64^{10}	0
AddRounkey	1	128	1	1408

Table III: Security analyses of the DAES and AES in terms of bit length (OSS: Operational Security Strength; ESLV: Effective security length of variables.)

4.3 Performance Analyses of the DAES

Experiments were performed using a computer with 11th Gen Intel Core i7-11700 2.5GHz CPU, 16 logical and 8 physical cores with 16GB of main memory and the operating system was Windows 10. The language used is C# to evaluate the performance of the AES and DAES in terms of their Key Schedule and Key and Parameter Schedules using BenchmarkDotNet given the following parameters: launchCount 22, iterationCount 50, and global setup after loading the plaintext file into memory.

Table IV lists the results of single-block encryption and various plaintext sizes. The time spent by the Key and Parameter Schedules is 204.948 times that of the Key Schedule, due to the production of D-boxes. Table V lists the time spent by running GBT1 and GBT2. The replacement of the GBT1 with the GBT2 to yield D_2-box–D_{10}-box not only improves the execution efficiency, but also ensures the security of D-boxes.

Table VI presents the encryption times consumed for a single-block encryption. In fact, the time-consumption ratio is only 1.2757. Table VII shows the encryption times on different plaintext sizes. For smaller sizes, the difference is significant. The ratio of DAES to AES was 1.8, due to the additional time required for the Key and Parameter Schedules of the DAES; the effect decreases as the plaintext sizes are higher. The ratio on 100 MB is 1.2782, which is not far away from that on a single block, 1.2757 (See Table VI).

Method	Time Consumed (µs)
Key Schedule (AES)	0.097
Key and Parameter Schedules (DAES)	19.88
DAES/AES	204.948

Table IV: The performance of the Key Schedule and the Key and Key and Parameter Schedules.

Method	Time consumed (µs)
GBT1	6.32
GBT2	0.23
GBT1/GBT2	27.47

Table V: The time consumed for the execution of GBT1 and GBT2 Algorithms.

Method	Time consumed (µs)
AES	0.5321
DAES	0.6788
DAES/AES	1.2757

Table VI: The time consumed for single block encryption.

Method / File Size	AES	DAES	DAES/AES
1 KB	0.0318	0.0573	1.8007
10 KB	0.3143	0.4239	1.3487
100 KB	3.3214	4.2697	1.2855
1 MB	34.5190	44.3513	1.2848
10 MB	331.1215	423.5060	1.2790
100 MB	3312.8377	4234.5784	1.2782

Table VII: The time spent given different plaintext sizes (ms).

5. Conclusions and Future Studies

In this paper, we propose the DAES, which inherits the confusion and diffusion characteristics of the AES, while in each transformation round: (1) D-box is employed to replace the S-box to increase the contribution corresponding to the 1683/30 bits of the ESLV for enhancing the security of the SubBytes operation; (2) Replacing ShiftRows operation with the DBR, which provides 64 variations. Now, we dare to say that the AES is one with 1/4 dynamics. and we upgraded it to a 3/4 dynamic one.

Furthermore, we lengthen of CK/PW to a range between 128 and 1024 bits to effectively resist brute-force attacks and enhance its user friendliness. The functionality of the Key and Parameter Schedules is expanded and adopt an irreversible algorithm, i.e., DSM, with an excellent avalanche effect to lower the correlation among keys and among parameters, thus effectively increasing system security.

Currently, the functions of computer hardware and software are improved quickly. 128-bit block cipher encryption mechanisms may soon be unsafe. The data delivered through commercial networks or the Internet is often longer than megabyte. Thus, a new cryptography system with a 1024-bit block is one of the research topics of our future studies. We will also formally derive behavior and reliability models for the DAES so that users can comprehend the behaviors and reliabilities of this system before using it. Furthermore, for a security system, validating it with a standard cryptographic evaluation tool, like NIST test suite, is necessary. These issues will be addressed in our future research.

References

[1] Dworkin, M.J., Turan, M.S. and Mouha, N. (2023, May). Advanced Encryption Standard (AES). National Institute of Standards and Technology (NIST) (pp.1-38). Retrieved from https://www.nist.gov/publications/advanced-encryption-standard-aes-0.

[2] Yang, F., Xu, M., Lei, W. and Lv, J. (2023, September). Artificial Intelligence Methods Applied to Catalytic Cracking Processes. Big Data Mining and Analytics, vol. 6, no. 3 (pp.361-380). Retrieved from https://www.sciopen.com/article/10.26599/BDMA.2023.9020002?issn=2096-0654/1000

[3] Yang, Z., Zolanvari, M. and Jain, R. (2023, March). A Survey of Important Issues in Quantum Computing and Communications. IEEE Communications Surveys & Tutorials, vol. 25, no. 2 (pp. 1059-1094). Retrieved from https://ieeexplore.ieee.org/document/10064036

[4] DeCusatis, C., Spillane, E. Foti, D. and O'Loughlin, M. (2024, October). Comparison of quantum computing algorithms for network traffic partitioning and mitigation of DoS and DDoS attacks," In Proc. of UEMCON (pp.0454-0460). Retrieved from https://www.semanticscholar.org/paper/Comparison-of-quantum-computing-algorithms-for-and-DeCusatis-Spillane/055ea15f7982d382a53383fbd56187257bcd0e99

[5] Nirbaya, M.H.M., Mahfud, A.Z., Bewasana, D.A.D. and Sunaringtyas, S.U. (2023, August). Metaverse-Based Learning Media to Increase Wireless Security Awareness," In Proc. ICE3IS, Yogyakarta, Indonesia. (pp. 175-180). Retrieved from https://ieeexplore.ieee.org/document/10335096

[6] Tong, J., Long, Y. and Liu, Q. (2021, November). A File Encryption System Based on Attribute Based Encryption. In Proc. CIS, Chengdu, China (pp.454-458). Retrieved from https://ieeexplore.ieee.org/document/9701774

[7] Wang, P., Zhang, Y. and Yang, J. (2018). Research and Design of AES Security Processor Model Based on FPGA. Procedia Computer Science, vol. 131 (pp.249-254). Retrieved from https://www.sciencedirect.com/science/article/pii/S1877050918305908

[8] Wang, Q., Wang, A., Wu, L. and Zhang, J. (2016, September). A New Zero Value Attack Combined Fault Sensitivity Analysis on Masked AES. Microprocessors and Microsystems, vol. 45 (pp. 355–362). Retrieved from https://www.sciencedirect.com/science/article/pii/S0141933116300837.

[9] Jithendra, K,B. and Shahana, T.K. (2020, March, updated on 2023, September). New Biclique Cryptanalysis on Full-Round PRESENT-80 Block Cipher. SN COMPUT. SCI. vol.1, no. 94. Retrieved from https://doi.org/10.1007/s42979-020-0103-z.

[10] Assafli, F.T. and Hashim, I.A. (2020, December). Generation and Evaluation of a New Time-Dependent Dynamic S-box Algorithm for AES Block Cipher Cryptosystems. In Proc. ICRIE 2020. Open access. Retrieved from file:///Users/user/Downloads/Generation_and_Evaluation_of_a_New_Time-Dependent_%20(1).pdf, doi:10.1088/1757-899X/978/1/012042.

[11] Abd-El-Atty, B, (2013, February). Efficient S-box construction based on quantum-inspired quantum walks with PSO algorithm and its application to image cryptosystem. Complex & Intelligent Systems, vol. 9 (pp. 4817–4835). Retrieved from https://link.springer.com/article/10.1007/s40747-023-00988-7.

[12] Saha, R., Geetha, G., Kumar, G. and Kim, T.H. (2018, November). RK-AES: an Improved Version of AES using a New Key Generation Process with Random Keys. Security and Communication Networks (pp. 1–11). Retrieved from https://doi.org/10.1155/2018/9802475.

[13] Luong, T. T. and Linh, H. D. (2014, February). On Generating New Key Dependent XOR Tables to Improve AES Security and Evaluating the Randomness of the Output of Block Ciphers. International Journal of Information and Computer Security, vol. 23, no 1 (pp.16-39). Retrieved from https://doi.org/10.1504/IJICS.2024.136710.

[14] Kazlauskas, K. (2015, February). An Algorithm for Key-dependent S-box Generation in Block Cipher System. Informatica, vol. 26 (pp. 51–65). Retrieved from https://dl.acm.org/doi/abs/10.5555/ios.INF1042

[15] Su, N., Zhang, Y. and Li, M. (2019, March). Research on Data Encryption Standard based on AES Algorithm in Internet of Things Environment. In Proc. ITNEC (pp.2071–2075). Retrieved from https://doi.org/10.1109/ITNEC.2019.8729488

[16] Huang, Y.L., Leu, F.Y., You, I. Su, R.Y. Su, P.H. and Chen, H.C. (2017, January). A 3D Encryption with Shifting Mapping Substitution Mechanism. In Proc. IEEE Annual Consumer Communications & Networking Conference. (pp.1–6). Retrieved from https://ieeexplore.ieee.org/document/8013422